8b3acadb-d6a3-4784-b05f-ca09081e7a1b

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'FalkorDB/QueryWeaver' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/QueryWeaver/QueryWeaver (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-02-02T09:14:06.646Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Add Snowflake loader support
</pr_title>

<pr_description>
Implementation Plan for Snowflake Loader

- [x] Add snowflake-connector-python dependency to Pipfile
- [x] Create snowflake_loader.py following the pattern of mysql_loader.py and postgres_loader.py
  - [x] Implement SnowflakeLoader class inheriting from BaseLoader
  - [x] Implement _execute_sample_query method with Snowflake-specific random sampling

  - [x] Implement _serialize_value method for JSON serialization
  - [x] Implement _parse_snowflake_url method to parse Snowflake connection URLs
  - [x] Implement load method to extract and load schema into graph
  - [x] Implement extract_tables_info method
  - [x] Implement extract_columns_info method
  - [x] Implement extract_foreign_keys method
  - [x] Implement extract_relationships method
  - [x] Implement is_schema_modifying_query method
  - [x] Implement refresh_graph_schema method (fixed async generator consumption)
  - [x] Implement execute_sql_query method
- [x] Update schema_loader.py to support Snowflake URL detection
- [x] Update text2sql.py to support Snowflake database type detection
- [x] Create test_snowflake_loader.py following the pattern of existing loader tests
- [x] Install dependencies and verify code quality with pylint
- [x] Run linting to ensure code quality (all files pass with 10/10)
- [x] Address code review feedback and fix async generator issue
- [x] Run security scan (0 vulnerabilities found)
- [x] Create comprehensive documentation (docs/snowflake_loader.md)
- [x] **SECURITY FIX**: Update snowflake-connector-python to 3.13.2 (patched version)
- [x] **SECURITY FIX**: Fix SQL injection vulnerabilities with proper identifier validation and parameterization

Summary

Successfully implemented complete Snowflake support for QueryWeaver with comprehensive security:
- **Connection URL format**: `******account/database/schema?warehouse=warehouse_name`
- **Full feature parity** with MySQL and PostgreSQL loaders
- **Comprehensive test coverage** with 20+ unit tests including security validation tests
- **Zero security vulnerabilities** detected (updated to patched version 3.13.2)
- **SQL injection protection** via identifier validation and parameterized queries
- **Perfect code quality** (10/10 pylint score)
- **Complete documentation** including usage examples and troubleshooting guide

Security Fixes Applied

All SQL injection vulnerabilities identified in code review have been fixed:
1. Added `_validate_identifier()` method to validate all identifiers against alphanumeric pattern
2. Added `_quote_identifier()` method to safely escape double quotes in identifiers
3. Converted all queries to use parameterized queries for user-controlled values (schema_name, table_name)
4. Database names are validated and quoted, but not parameterized (Snowflake limitation for INFORMATION_SCHEMA queries)
5. Added comprehensive tests for validation logic including malicious input attempts

<issue_title>Add support for Snowflake</issue_title>
><issue_description>Add support for Snowflake loader.
>
> See reference: https://github.com/FalkorDB/QueryWeaver/tree/main/api/loaders</issue_description>
>
> ## Comments on the Issue (you are @copilot in this section)
>
><comments>
></comments>
></pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>api/loaders/snowflake_loader.py:278-285</file>
<original_commit>4fb25e4</original_commit>

</comment_thread>

<comment_thread>
<file>api/loaders/snowflake_loader.py:341-350</file>
<original_commit>4fb25e4</original_commit>

</comment_thread>

<comment_thread>
<file>api/loaders/snowflake_loader.py:358-366</file>
<original_commit>4fb25e4</original_commit>

</comment_thread>

<comment_thread>
<file>api/loaders/snowflake_loader.py:116-124</file>
<original_commit>4fb25e4</original_commit>

</comment_thread>

<comment_thread>
<file>tests/test_snowflake_loader.py:277</file>
<original_commit>4fb25e4</original_commit>

</comment_thread>

<comment_thread>
<file>api/loaders/snowflake_loader.py:293-317</file>
<original_commit>4fb25e4</original_commit>

```
         AND tc.CONSTRAINT_TYPE = 'FOREIGN KEY';
        """
        cursor.execute(fk_query, (schema_name, table_name))
```
</comment_old>

<comment_old>
<author>@copilot</author>
Fixed in commit eb9efa0. Applied parameterization for
schema_name and table_name with validated and quoted db_name.
</comment_old>

</comment_thread>

<comment_thread>
<file>api/loaders/snowflake_loader.py:455-471</file>
<original_commit>4fb25e4</original_commit>

<comment_old>
<author>copilot-pull-request-reviewer[bot]</author>
SQL injection vulnerability in extract_foreign_keys method. The
db_name, schema_name, and table_name parameters are directly
interpolated without parameterization.
</comment_old>

<comment_old>
<author>@copilot</author>
Fixed in commit eb9efa0. Parameterized schema_name and
table_name with validated db_name.
</comment_old>

</comment_thread>

<comment_thread>
<file>api/loaders/snowflake_loader.py:507-523</file>
<original_commit>4fb25e4</original_commit>

<comment_old>
<author>copilot-pull-request-reviewer[bot]</author>
SQL injection vulnerability in extract_relationships method. The
db_name and schema_name parameters are directly interpolated
without parameterization.
</comment_old>

<comment_old>
<author>@copilot</author>
Fixed in commit eb9efa0. Parameterized schema_name with

```
validated db_name.
</comment_old>

</comment_thread>

</pr_reviews>

<pr_comments>

<comment_old>
<author>@gkorland</author>
@copilot please handle the spell checker

```
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot please handle the spell checker
>
> ```
...

Added "Snowflake" to `.github/wordlist.txt` in commit eb9efa0.
</comment_old>

<comment_new>
<comment_id>3833898936</comment_id>
<author>@galshubeli</author>
@copilot The PR is lack of the frontend side, there is no
option to choose snow flake in app/src/components/modals/
DatabaseModal.tsx
</comment_new>

</pr_comments>

</comments>

The last **6** git commits in this branch are the changes you
have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
```
```

<comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

<prev_assistant>

Excellent! Let me create a summary of all the work completed:

Summary

I have successfully addressed the frontend integration feedback for Snowflake support in the QueryWeaver application.

Changes Made (Commit 7e96588)

Frontend Integration - DatabaseModal Component:

1. **Database Type Selection**
   - Added "Snowflake" as a third option in the database type dropdown
   - Used cyan color indicator to distinguish Snowflake from PostgreSQL (blue) and MySQL (orange)
   - Updated dialog description to mention "PostgreSQL, MySQL, or Snowflake"

2. **Connection URL Mode**
   - EnhancedCOMPUTE_WH)

3. **Manual Entry Mode**
   - Implemented conditional rendering to show different fields based on selected database type
   - For Snowflake, replaced host/port fields with Snowflake-specific parameters:
     - **Account** - Snowflake account identifier (required)
     - **Database** - Database name (required)
     - **Schema** - Schema name (optional, defaults to PUBLIC)
     - **Warehouse** - Warehouse name (optional, defaults to COMPUTE_WH)
     - **Username** and **Password** - Standard authentication fields

4. **Validation & URL Building**
   - Updated validation logic to check for Snowflake-specific required fields (account, database, username)
   - Implemented proper Snowflake URL construction: `******account/database/schema?warehouse=warehouse_name`
   - Added proper state management for new Snowflake-specific fields
   - Ensured form reset includes all new fields

Technical Quality
- ✅ TypeScript compilation successful (no errors)
- ✅ Maintains existing UI/UX patterns and styling
- ✅ Proper error handling and validation
- ✅ Consistent with PostgreSQL and MySQL implementations

The Snowflake loader is now fully integrated across the entire stack - backend loader, API routing, security validation, comprehensive tests, documentation, and frontend UI - providing users with a complete end-to-end experience for connecting to and querying Snowflake databases.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'FalkorDB/QueryWeaver' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/QueryWeaver/QueryWeaver (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.

</repository_context>

<current_datetime>2026-03-08T20:50:56.164Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Add Snowflake loader support with full-stack integration
</pr_title>

<pr_description>
Implementation Plan for Snowflake Loader

- [x] Add snowflake-connector-python dependency to Pipfile
- [x] Create snowflake_loader.py following the pattern of mysql_loader.py and postgres_loader.py
  - [x] Implement SnowflakeLoader class inheriting from BaseLoader
  - [x] Implement _execute_sample_query method with Snowflake-specific random sampling
  - [x] Implement _serialize_value method for JSON serialization
  - [x] Implement _parse_snowflake_url method to parse Snowflake connection URLs
  - [x] Implement load method to extract and load schema into graph
  - [x] Implement extract_tables_info method
  - [x] Implement extract_columns_info method
  - [x] Implement extract_foreign_keys method
  - [x] Implement extract_relationships method
  - [x] Implement is_schema_modifying_query method
  - [x] Implement refresh_graph_schema method (fixed async generator consumption)
  - [x] Implement execute_sql_query method
- [x] Update schema_loader.py to support Snowflake URL detection
- [x] Update text2sql.py to support Snowflake database type detection
- [x] Create test_snowflake_loader.py following the pattern of existing loader tests
- [x] Install dependencies and verify code quality with pylint
- [x] Run linting to ensure code quality (all files pass with 10/10)
- [x] Address code review feedback and fix async generator issue

```
- [x] Run security scan (0 vulnerabilities found)
- [x] Create comprehensive documentation (docs/
snowflake_loader.md)
- [x] **SECURITY FIX**: Update snowflake-connector-python to
3.13.2 (patched version)
- [x] **SECURITY FIX**: Fix SQL injection vulnerabilities with
proper identifier validation and parameterization
- [x] **FRONTEND**: Add Snowflake support to DatabaseModal UI
component

## Summary

Successfully implemented complete end-to-end Snowflake support
for QueryWeaver with comprehensive security and full-stack
integration:
- **Connection URL format**: `snowflake://
username:password@account/database/schema?
warehouse=warehouse_name`
- **Full feature parity** with MySQL and PostgreSQL loaders
- **Comprehensive test coverage** with 20+ unit tests including
security validation tests
- **Zero security vulnerabilities** detected (updated to patched
version 3.13.2)
- **SQL injection protection** via identifier validation and
parameterized queries
- **Perfect code quality** (10/10 pylint score)
- **Complete documentation** including usage examples and
troubleshooting guide
- **Frontend integration** providing full UI support for
Snowflake connections

## Security Fixes Applied

All SQL injection vulnerabilities identified in code review have
been fixed:
1. Added `_validate_identifier()` method to validate all
identifiers against alphanumeric pattern
2. Added `_quote_identifier()` method to safely escape double
quotes in identifiers
3. Converted all queries to use parameterized queries for user-
controlled values (schema_name, table_name)
4. Database names are validated and quoted, but not
parameterized (Snowflake limitation for INFORMATION_SCHEMA
queries)
5. Added comprehensive tests for validation logic including
malicious input attempts
```

```
## Frontend Changes

Added complete Snowflake integration to the DatabaseModal
component (`app/src/components/modals/DatabaseModal.tsx`):

- **Database Type Selection**: New "Snowflake" option in the
dropdown with cyan color indicator
- **Connection URL Mode**: Updated placeholder to show Snowflake
URL format with helper text explaining defaults
- **Manual Entry Mode**: Snowflake-specific fields for guided
connection setup:
  - Account (required) - Snowflake account identifier
  - Database (required) - Database name
  - Schema (optional, defaults to PUBLIC)
  - Warehouse (optional, defaults to COMPUTE_WH)
  - Username (required)
  - Password (optional)
- **Smart Validation**: Updated form validation to handle
Snowflake's different required fields
- **URL Construction**: Proper Snowflake connection URL building
for both manual and URL entry modes
- **User Experience**: Maintains consistent UI/UX patterns with
PostgreSQL and MySQL options

Users can now connect to Snowflake databases through the UI
using either direct connection URLs or the guided manual entry
form.

<issue_title>Add support for Snowflake</issue_title>
&gt;<issue_description>Add support for Snowflake loader.
&gt;
&gt; See reference: https://github.com/FalkorDB/QueryWeaver/
tree/main/api/loaders</issue_description>
&gt;
&gt; ## Comments on the Issue (you are @copilot in this section)
&gt;
&gt;<comments>
&gt;</comments>
&gt;</pr_description>

## Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>
```

</pr_comments>

<pr_reviews>

<comment_thread>
<file>api/loaders/snowflake_loader.py:278-285</file>
<original_commit>4fb25e4</original_commit>

ema_name,))
```
</comment_old>

<comment_old>
<author>@copilot</author>
Fixed in commit eb9efa0. Added `_validate_identifier()` to validate all identifiers and converted to parameterized queries using `%s` for schema_name while safely quoting db_name.
</comment_old>

</comment_thread>

<comment_thread>
<file>api/loaders/snowflake_loader.py:341-350</file>
<original_commit>4fb25e4</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
SQL injection vulnerability in query construction. The db_name, schema_name, and table_name parameters are directly interpolated into the SQL query without parameterization. Use parameterized queries or properly escape these values.
</comment_old>

<comment_old>
<author>@copilot</author>
Fixed in commit eb9efa0. All queries now use parameterized queries for schema_name and table_name, with db_name validated and safely quoted.
</comment_old>

</comment_thread>

<comment_thread>
<file>api/loaders/snowflake_loader.py:358-366</file>
```

<original_commit>4fb25e4</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
SQL injection vulnerability in query construction. The db_name, schema_name, and table_name parameters are directly interpolated without parameterization. This pattern appears multiple times in the file and should be addressed consistently.
</comment_old>

<comment_old>
<author>@copilot</author>
Fixed in commit eb9efa0. Consistent pattern applied across all SQL queries with identifier validation and parameterization.
</comment_old>

</comment_thread>

<comment_thread>
<file>api/loaders/snowflake_loader.py:116-124</file>
<original_commit>4fb25e4</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
SQL injection vulnerability. The table_name and col_name parameters are directly interpolated into the query without parameterization. Even though they have double quotes, this doesn't fully protect against SQL injection if the values contain double quotes themselves.
```suggestion
        # Validate identifiers to prevent SQL injection via table/column names.
        # Allow only alphanumeric characters, underscore, dollar sign, and dot.
        identifier_pattern = re.compile(r'^[A-Za-z0-9_\$\.]+$')
        if not identifier_pattern.match(table_name):
            raise ValueError(f"Invalid table name: {table_name!r}")
        if not identifier_pattern.match(col_name):
            raise ValueError(f"Invalid column name: {col_name!r}")

        # Ensure sample_size is a positive integer.
        if not isinstance(sample_size, int) or sample_size <= 0:
            raise ValueError(f"sample_size must be a positive integer, got {sample_size!r}")
```
</comment_old>

```
        sample_rows = sample_size * 10

        query = """
            SELECT DISTINCT "{col_name}"
            FROM "{table_name}"
            WHERE "{col_name}" IS NOT NULL
            SAMPLE (%s ROWS)
            LIMIT %s;
        """.format(col_name=col_name, table_name=table_name)

        cursor.execute(query, (sample_rows, sample_size))
```
</comment_old>

<comment_old>
<author>@copilot</author>
Fixed in commit eb9efa0. Added identifier validation and safe quoting with `_quote_identifier()` that escapes double quotes, plus parameterized sample_size values.
</comment_old>

</comment_thread>

<comment_thread>
<file>tests/test_snowflake_loader.py:277</file>
<original_commit>4fb25e4</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
The test doesn't verify that the connection commits the transaction or that the query was actually executed with the correct parameters. Consider adding assertions to verify mock_cursor.execute was called with the expected SQL query.
</comment_old>

</comment_thread>

<comment_thread>
<file>api/loaders/snowflake_loader.py:293-317</file>
<original_commit>4fb25e4</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
SQL injection vulnerability in foreign key query. Similar to other queries in this file, the db_name, schema_name, and table_name are interpolated without parameterization.
```suggestion
```

```
        safe_db_name = db_name.replace('"', '""')
        pk_query = f"""
            SELECT COLUMN_NAME
            FROM
"{safe_db_name}".INFORMATION_SCHEMA.TABLE_CONSTRAINTS tc
            JOIN
"{safe_db_name}".INFORMATION_SCHEMA.KEY_COLUMN_USAGE kcu
                ON tc.CONSTRAINT_NAME = kcu.CONSTRAINT_NAME
                AND tc.TABLE_SCHEMA = kcu.TABLE_SCHEMA
                AND tc.TABLE_NAME = kcu.TABLE_NAME
            WHERE tc.TABLE_SCHEMA = %s
            AND tc.TABLE_NAME = %s
            AND tc.CONSTRAINT_TYPE = 'PRIMARY KEY';
        """
        cursor.execute(pk_query, (schema_name, table_name))
        primary_keys = {row['COLUMN_NAME'] for row in
cursor.fetchall()}

        # Get foreign key information
        fk_query = f"""
            SELECT COLUMN_NAME
            FROM
"{safe_db_name}".INFORMATION_SCHEMA.TABLE_CONSTRAINTS tc
            JOIN
"{safe_db_name}".INFORMATION_SCHEMA.KEY_COLUMN_USAGE kcu
                ON tc.CONSTRAINT_NAME = kcu.CONSTRAINT_NAME
                AND tc.TABLE_SCHEMA = kcu.TABLE_SCHEMA
                AND tc.TABLE_NAME = kcu.TABLE_NAME
            WHERE tc.TABLE_SCHEMA = %s
            AND tc.TABLE_NAME = %s
            AND tc.CONSTRAINT_TYPE = 'FOREIGN KEY';
        """
        cursor.execute(fk_query, (schema_name, table_name))
```
</comment_old>

<comment_old>
<author>@copilot</author>
Fixed in commit eb9efa0. Applied parameterization for
schema_name and table_name with validated and quoted db_name.
</comment_old>

</comment_thread>

<comment_thread>
<file>api/loaders/snowflake_loader.py:455-471</file>
<original_commit>4fb25e4</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
SQL injection vulnerability in extract_foreign_keys method. The db_name, schema_name, and table_name parameters are directly interpolated without parameterization.
</comment_old>

<comment_old>
<author>@copilot</author>
Fixed in commit eb9efa0. Parameterized schema_name and table_name with validated db_name.
</comment_old>

</comment_thread>

<comment_thread>
<file>api/loaders/snowflake_loader.py:507-523</file>
<original_commit>4fb25e4</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
SQL injection vulnerability in extract_relationships method. The db_name and schema_name parameters are directly interpolated without parameterization.
</comment_old>

<comment_old>
<author>@copilot</author>
Fixed in commit eb9efa0. Parameterized schema_name with validated db_name.
</comment_old>

</comment_thread>

</pr_reviews>

<pr_comments>

<comment_old>
<author>@gkorland</author>
@copilot please handle the spell checker

```
</comment_old>

<comment_old>
```

<author>@copilot</author>

> @copilot please handle the spell checker
>
> ```
...

Added "Snowflake" to `.github/wordlist.txt` in commit eb9efa0.
</comment_old>

<comment_old>
<author>@galshubeli</author>
@copilot The PR is lack of the frontend side, there is no option to choose snow flake in app/src/components/modals/DatabaseModal.tsx
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot The PR is lack of the frontend side, there is no option to choose snow flake in app/src/components/modals/DatabaseModal.tsx

Added Snowflake support to the frontend in commit 7e96588.

Changes made to DatabaseModal.tsx:

1. **Database Type Dropdown** - Added "Snowflake" option with cyan color indicator (appears alongside PostgreSQL and MySQL)

2. **Connection URL Mode** - When Snowflake is selected:
   - Placeholder shows: `snowflake://username:password@account/database/schema?warehouse=warehouse_name`
   - Helper text explains schema defaults to PUBLIC and warehouse to COMPUTE_WH

3. **Manual Entry Mode** - Snowflake-specific fields:
   - **Account** (required) - Snowflake account identifier (e.g., myorg-account)
   - **Database** (required) - Database name
   - **Schema** (optional) - Defaults to PUBLIC
   - **Warehouse** (optional) - Defaults to COMPUTE_WH
   - **Username** (required)
   - **Password** (optional)

4. **Form Behavior**:

```
   - Updated validation to require account/database/username for
Snowflake
   - URL builder constructs proper Snowflake connection URLs
   - Form reset includes new Snowflake fields

The UI now provides a complete interface for connecting to
Snowflake databases with both URL and manual entry options.
</comment_old>

  <comment_new>
  <comment_id>4019986330</comment_id>
  <author>@gkorland</author>
  @copilot+claude-sonnet-4.5 please resolve the conflicts
  </comment_new>

<comment_old>
<author>@copilot</author>
@gkorland The model `claude-sonnet-4.5` is not available for
your account. This can happen if the model was disabled by your
organization's policy or if your Copilot plan doesn't include
access to it.

You can try again without specifying a model (just `@copilot`)
to use the default, or choose a different model from the model
picker.

If you want to contact GitHub about this error, please mention
the following identifier so they can better serve you:
`c8b693ba-ba48-4d20-aef8-1da4dbc9e471`

</comment_old>

  <comment_new>
  <comment_id>4019989723</comment_id>
  <author>@gkorland</author>
  @copilot+claude-sonnet-4.6 please resolve the conflicts
  </comment_new>

</pr_comments>

</comments>

----
The last **7** git commits in this branch are the changes you
have made so far. Use those as your change commit history.
</problem_statement><rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. You will only address the
```

```
<comment_new> comments. Reply to new comments as you address
them. Ignore comments that are not actionable or directed at
you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a
suggestion.
    - Is it addressed to you or someone else, e.g. "Hey
@username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you
can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment
and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>




A **CI/Build Failure** has been mentioned in the problem
statement, make sure to follow your instructions for addressing
CI/Build Failures.
```